Computational Identification of High-Altitude Molecules Using Molecular Networking and Liquid Chromatography High-Resolution Mass Spectrometry (LC-HRMS): Developing Software by Chemists, for Chemists

Independent Study Thesis

Presented in Partial Fulfillment of the Requirements for
the Degree Bachelor of Arts in the
Departments of Chemistry & Computer Science at The College of
Wooster

by
Raelyn Brooks
The College of Wooster
2027

Advised by:

Jennifer Faust (Chemistry)

John Musgrave (Computer Science)



THE COLLEGE OF
WOOSTER

© 2027 by Raelyn Brooks

Abstract

Include a short summary of your thesis, including any pertinent results. This section is *not* optional, and the reader should be able to learn the meat of your thesis by reading this (short) section.

This work is dedicated to the future generations of Wooster students.

Acknowledgments

No part of my journey at The College of Wooster has been accomplished alone. Behind every class, project, difficult conversation, and moment of growth are people who have supported me, challenged me, and reminded me why I started. This Independent Study is no exception. I am grateful to everyone who helped me become not only a better student and researcher, but also a better person.

First, I want to thank my Senior Independent Study advisors, Dr. Faust and Dr. Musgrave, for guiding me through a project that brings chemistry and computer science together. Dr. Faust, you have been one of my biggest champions. Thank you for believing in me, advocating for me, and helping me see what I am capable of even when I have struggled to see it myself. Dr. Musgrave, thank you for your guidance, patience, and willingness to work through the challenges of turning an interdisciplinary idea into a meaningful computer science project. I am also grateful to Dr. Paul Edmiston for bringing the original molecular-networking problem to my attention and giving me a reason to explore how the two disciplines I love could work together.

To the Computer Science Department, thank you for teaching me how to think critically, build things, and persist when the first solution fails. Dr. Heather Guarnera, thank you for always helping me find a way to make the impossible seem possible. Dr. Drew Guarnera, as my first computer science professor, you helped build a foundation that continues to shape my work. Dr. Palmer, thank you for contributing to my growth as a programmer and computer scientist. I am grateful to the rest of the department as well, including the faculty and staff whose time, encouragement, and teaching helped me find my place in the field.

To the Chemistry Department, thank you for making science something I wanted not only to study, but to spend my life exploring. Dr. Martin, thank you for your guidance as my academic advisor and for supporting my path across two majors. Dr. Snider, thank you for the difficult but necessary conversation that helped me grow. Dr. Bonvallet, thank you for making lectures something to look forward to. Dr. Gallo, thank you for helping me fall in love with chemistry. Dr. Baker, thank you for being one of my favorite professors and one of my favorite people to stop and say hello to. Dr. Feierabend, thank you for pushing me to be great, especially in the moments when I did not fully believe I could be. I am grateful to every other member of the Chemistry Department who shared their knowledge, offered encouragement, or made the department feel like a community.

To my best friends, Kate Carson, Eliza Davis, and Brooke Bradley, thank you for being part of my life through the busy days, the stressful weeks, and the moments worth celebrating. Your friendship has meant more to me than I can put into a single paragraph. Thank you for giving me people I could laugh with, rely on, and be myself around.

To my brothers of Beta Kappa Phi, thank you for giving me a brotherhood and a home within Wooster. I especially want to recognize Shea McLaren, Cooper Norwell, Gabriel Jones, Nathan Krieg, Parth, Isaac Prince, Jackson Ferrell and Matthew Dalton, my littles, Hayden Jacoby and Peter Dentzer, my grand big and big. Each of you has helped shape what brotherhood, responsibility, and friendship mean to me. I am proud of the memories we have made and the people we have become together. The first and finest will always hold a special place in my heart.

To the Residence Life Office, thank you for the opportunity to grow as a leader and learn what it means to show up for a community. I also want to thank Maddie Cuckow, my former AD and now Greek Life Director, for being part of that journey and for the support and guidance you have given me along the way.

I want to give a special thank-you to Lauren Richmond, who has been an important part of my life and personal growth throughout the years. Although life has taken us in different directions, I will always be grateful for the love, encouragement, and support she has given me. The memories we created, the challenges we faced, and the experiences we shared have shaped me in ways I will always appreciate. Her impact on my journey extends far beyond our time together, and I will always value the role she played in helping me become the person I am today.

To Ty Reynolds, Doug Slovenkay, Ethan Hunt, Taryn Brady, Saveer Patil, Levi Monroe, Ben Kimble, Madeline Eaton, and Sofia Wolken, thank you for being part of my college experience. Whether through conversation, friendship, encouragement, or simply being present along the way, you have each contributed to memories and experiences I will carry with me beyond Wooster.

To Fr. David, thank you for walking alongside me during my journey toward baptism. Your guidance helped me through a deeply personal part of my life, and I am grateful for the time, care, and faith you shared with me.

Most importantly, I thank my family and siblings, especially my seven sisters and six brothers. You are part of the reason I care so deeply about using what I learn to make a difference. Thank you for the love, sacrifices, patience, and encouragement that have carried me through every stage of my education. Whatever I accomplish, I will always carry my family with me.

Finally, to everyone whose name may not appear here but who has offered a kind word, a difficult lesson, a new opportunity, or a reason to keep going, thank you. This work may have my name on its cover, but the person who completed it was shaped by all of you.

Vita

Raelyn M. Brooks is a candidate for the Bachelor of Arts degree in Chemistry and Computer Science at The College of Wooster, with an anticipated graduation date of May 2027. His academic interests lie at the intersection of computational chemistry, software engineering, and scientific research. He is particularly interested in developing computational tools that allow scientists to investigate complex chemical problems more efficiently and reliably.

During his undergraduate studies, Brooks pursued interdisciplinary research combining chemistry and computer science. His Senior Independent Study, advised by Dr. Jennifer Faust and Dr. John Musgrave, focuses on developing molecular-networking software to support the investigation of unknown molecular features in remote environmental samples using liquid chromatography–high- resolution mass spectrometry (LC-HRMS). His previous research includes developing a Python-based cheminformatics pipeline using RDKit to explore multistep organic synthesis pathways across large chemical reaction datasets.

Brooks has gained professional experience in software engineering and cloud computing. As a Software Developer Intern at the Federal Reserve Bank of St. Louis, he contributed to production software development, automated infrastructure, and security remediation. He has also worked as an Associate Cloud Engineer at GoHub LLC, developing cloud infrastructure and software for a transportation platform. Additionally, he has served as a coding instructor at iD Tech, teaching programming, software development, and problem-solving to hundreds of students.

Outside the classroom, Brooks has been actively involved in campus leadership and community engagement. He has served as President of Beta Kappa Phi, participated in Black & Code, and contributed to student life through Residence Life and other campus organizations. These experiences have strengthened his commitment to mentorship, leadership, and creating opportunities for others.

Following graduation, Brooks intends to pursue a career in software engineering, particularly in developing computational tools for chemistry and scientific research. He also hopes to pursue graduate study in chemistry, computer science, or an interdisciplinary field connecting the two disciplines.

Fields of Study Major field: Chemistry & Computer Science

Specialization: Computational Chemistry, Molecular Networking, and Liquid Chromatography–High-Resolution Mass Spectrometry (LC-HRMS)

Contents

Abstract iii

Dedication iv

Acknowledgments v

Vita vii

Contents viii

List of Figures x

List of Tables xi

List of Listings xii

Preface xiii

CHAPTER PAGE

1 Introduction 1
- 1.1 Research Problem and Motivation . . . 1
- 1.2 Environmental and Chemical Background . . . 2
  - 1.2.1 Long-Range Transport of Organic Contaminants . . . 2
  - 1.2.2 High-Altitude and Remote Surface-Water Environments . . . 2
  - 1.2.3 Previous Wooster Research and the Identification Gap . . . 2
- 1.3 LC-HRMS, Suspect Screening, and Non-Targeted Analysis . . . 3
  - 1.3.1 LC-HRMS and Tandem Mass Spectrometry . . . 3
  - 1.3.2 Molecular Features Versus Identified Compounds . . . 3
  - 1.3.3 Suspect Screening and Non-Targeted Analysis . . . 3
  - 1.3.4 Limitations of Current Identification Approaches . . . 4
- 1.4 Molecular Networking and Theoretical Foundations . . . 4
  - 1.4.1 MS/MS Fragmentation and Spectral Similarity . . . 4
  - 1.4.2 Cosine Similarity and Molecular-Network Construction . . . 4
  - 1.4.3 Graph Theory: Nodes, Edges, and Connected Components . . . 5
  - 1.4.4 Network Visualization and Interpretation . . . 5
  - 1.4.5 Limitations of Spectral Similarity as Evidence of Chemical Identity . . . 5
- 1.5 Existing Software and Research Gap . . . 6
  - 1.5.1 MS-DIAL and GNPS in the Existing Workflow . . . 6
  - 1.5.2 Documented Limitations in the Wooster Research Workflow . . . 6
  - 1.5.3 The Need for Software Designed Around Chemists' Needs . . . 6
- 1.6 Research Question and Specific Aims . . . 7
- 1.7 Project Scope and Thesis Contributions . . . 7
  - 1.7.1 Project Scope . . . 7
  - 1.7.2 Expected Computer Science Contributions . . . 8
  - 1.7.3 Expected Chemistry Contributions . . . 8
  - 1.7.4 Organization of the Thesis . . . 8

2 Methodology, Research Design, and Software Development 9
2.1 Overall Research Design and Workflow . . . 9
2.2 Environmental Samples and Chemistry Experimental Methods . . . 9
2.3 Upstream Data Processing and Chemical Annotation . . . 9
2.4 Software Requirements and System Architecture . . . 9
2.5 Spectral Comparison and Similarity Calculation . . . 9
2.6 Molecular-Network Construction . . . 9
2.7 User Interface Design and Interaction . . . 9
2.8 Software Implementation and Reproducibility . . . 9
2.9 Software Evaluation and Testing Methods . . . 9
2.10 Chemical Case-Study Design . . . 9

3 Results and Data Analysis 10
3.1 Environmental Dataset and Feature Summary . . . 10
3.2 Software Implementation Results . . . 10
3.3 Computational Correctness and Validation . . . 10
3.4 Software Performance and Workflow Evaluation . . . 10
3.5 Molecular-Network Structure and Characteristics . . . 10
3.6 Chemical Case Study 1: Annotated Molecular Features . . . 10
3.7 Chemical Case Study 2: Unidentified Molecular Features . . . 10
3.8 Summary of Principal Results . . . 10

4 Discussion 11
4.1 Interpretation and Integration of Results . . . 11
4.2 Computational Contributions and Design Trade-Offs . . . 11
4.3 Chemical Significance and Comparison with Literature . . . 11
4.4 Study and Implementation Limitations . . . 11
4.5 Implications for Future Environmental HRMS Research . . . 11

5 Conclusions and Future Work 12
5.1 Summary of the Research . . . 12
5.2 Principal Computer Science Contributions . . . 12
5.3 Principal Chemistry Contributions . . . 12
5.4 Answer to the Central Research Question . . . 12
5.5 Future Work . . . 12
5.6 Final Remarks . . . 12

APPENDIX PAGE

A Typesetting Mathematical Formulae 13
A.1 General . . . 13
A.2 Grouping in Math Mode . . . 15
A.3 Building Blocks of a Mathematical Formula . . . 15
A.4 Math Spacing . . . 18
A.5 Vertically Aligned Material . . . 19
A.6 Math Font Size . . . 21
A.7 Theorems, Laws, ... 22
A.8 Bold symbols . . . 23
A.9 List of Mathematical Symbols . . . 23

B Examples of Java Code 30

C C++ Examples 32

D Cover and Copyright Examples 34

Afterword 36

Index 38

List of Figures

Figure Page

D.1 Options for the IS Cover Page 34
(a) Grayscale Scottie Mascot 34
(b) Grayscale Wooster 'W' 34
D.2 woostercopytight option 35

List of Tables

Table Page

A.1 Math Mode Accents. . . . 24
A.2 Lowercase Greek Letters. . . . 24
A.3 Uppercase Greek Letters. . . . 24
A.4 Binary Relations. . . . 25
A.5 Binary Operators. . . . 25
A.6 BIG Operators. . . . 25
A.7 Arrows. . . . 26
A.8 Delimiters. . . . 26
A.9 Large Delimiters. . . . 26
A.10 Miscellaneous Symbols. . . . 26
A.11 Non-Mathematical Symbols. . . . 27
A.12 AMS Delimiters. . . . 27
A.13 AMS Greek and Hebrew. . . . 27
A.14 AMS Binary Relations. . . . 27
A.15 AMS Binary Relations Continued. . . . 27
A.16 AMS Arrows. . . . 28
A.17 AMS Negated Binary Relations and Arrows. . . . 28
A.18 AMS Binary Operators. . . . 29
A.19 AMS Miscellaneous. . . . 29
A.20 Math Alphabets. . . . 29

List of Listings

Listing Page

B.1 Clock Code 30
B.2 Consumer 31
B.3 EvilEmpire Code 31

C.1 Motion Class 32
C.2 Plotter Class 32
C.3 Simulation Class 33

Preface

I have always considered myself a problem solver. Whether I am writing code, working through a chemical reaction, or trying to understand why something does not work as expected, I am drawn to problems that require patience, creativity, and a willingness to approach things differently. This is ultimately what attracted me to both chemistry and computer science. Chemistry has taught me to think beyond what I can immediately observe and to consider the countless interactions that explain the world around us. Computer science has taught me that failure is often part of progress, and that solving a problem sometimes requires starting over, questioning an assumption, or finding an entirely different approach. Together, these disciplines have shaped how I think, how I learn, and the kind of work I hope to pursue.

The inspiration for this Independent Study came from a conversation with Dr. Paul Edmiston, who introduced me to a challenge involving molecular-networking software. His students were exploring how molecular networking could help identify chemically modified substances relevant to forensic toxicology and support investigations involving drug-facilitated crimes. However, understanding the computational processes behind the software and evaluating the reliability of its results presented a challenge. Dr. Edmiston needed someone willing to look beneath the surface of the software and understand how it worked. As a brother to seven sisters, the opportunity to contribute to research with potential implications for personal safety and justice resonated with me deeply. Beyond its scientific importance, I recognized a chance to apply my knowledge of chemistry and computer science to a problem that could make a meaningful difference in people's lives.

What made this opportunity especially exciting was that it brought together the two fields I had spent my undergraduate career studying. I have never viewed chemistry and computer science as competing interests. Instead, I see them as disciplines that strengthen one another. Chemistry presents complicated questions about the world, while computer science provides tools for investigating those questions at a scale and level of detail that would otherwise be difficult to achieve. Although the research eventually developed into a project centered on molecular networking and the identification of unknown features in remote environmental samples, that initial conversation remained central to my motivation: I wanted to understand how the software worked and how it could be made more useful and reliable for the scientists who depend on it.

My experiences outside the classroom have further shaped that goal. During my internship as a software developer at the Federal Reserve Bank of St. Louis, I learned the importance of writing dependable software, testing my work, and understanding the consequences of decisions made within complex systems. Working with cloud infrastructure and software architecture has shown me how careful design affects reliability and long-term usefulness. As a coding instructor, I have taught hundreds of students how to approach difficult programming problems, often reminding them of the same lesson I continue to learn myself: not understanding something immediately does not mean it cannot be understood. My previous computational chemistry work, including developing tools to explore multistep organic synthesis using large reaction datasets, also helped me recognize how powerful software can become when it is built with a clear understanding of the science it supports.

After graduation, I hope to pursue a career developing software for chemists and other scientific researchers. I want to build tools that do more than produce results. I want researchers to understand those results, explore the evidence behind them, and have confidence in the computational processes they use. This Independent Study represents my first substantial attempt to bring that ambition into practice. It has also challenged me to recognize that being both a chemist and a computer scientist does not mean having every answer. Instead, it means being willing to learn from both disciplines, recognize the limitations of each, and do the work necessary to connect them.

I would not have reached this point without the faculty members who have challenged, supported, and believed in me throughout my time at The College of Wooster.

Within the Computer Science Department, I want to thank Dr. Heather Guarnera for consistently helping me find ways to make what once seemed impossible feel achievable. Your encouragement has pushed me to think bigger about what I am capable of accomplishing. I thank Dr. Drew Guarnera, my first computer science professor, for helping establish the foundation upon which so much of my work now rests. I am also grateful to Dr. Palmer for contributing to my growth as a computer scientist and to Dr. Musgrave, my Senior Independent Study advisor, for guiding me through the computational challenges of this interdisciplinary project.

Within the Chemistry Department, I owe a special thank-you to Dr. Faust, my Senior Independent Study advisor and one of my greatest champions. Your belief in my abilities, your encouragement, and your willingness to help me grow have meant more than I can adequately express. I thank Dr. Martin, my academic advisor, for helping guide my academic journey and supporting my interests across two demanding disciplines. Dr. Snider, thank you for the difficult but necessary conversation that helped me recognize where I needed to grow. Sometimes the advice we need most is not the advice we expect to hear. I thank Dr. Bonvallet for being a lecturer whose teaching has consistently made learning chemistry engaging. I thank Dr. Gallo for

helping me fall in love with chemistry and recognize just how much there is to discover within it. Dr. Baker, thank you for being one of my favorite professors and for the simple kindness of always being someone I look forward to greeting. Finally, Dr. Feierabend, thank you for continuing to challenge me to pursue excellence, particularly during moments when I have struggled to recognize my own potential.

I also want to thank Dr. Edmiston for presenting the problem that ultimately helped me discover the direction I want to take my career. Sometimes the most important opportunities begin with a conversation and a question that nobody has fully answered.

This Independent Study is more than a requirement for graduation. It represents the intersection of my interests, my experiences, and the people who have helped shape my education. More importantly, it is a beginning. I entered college wanting to understand how things work. I hope to leave having taken my first meaningful steps toward building things that help others understand the world.

CHAPTER 1

Introduction

1.1 Research Problem and Motivation

Organic contaminants are an important environmental chemistry concern because chemicals released through industrial activity, agriculture, and consumer-product use can move beyond their original sources. As a result, chemicals associated with human activity may be detected in environments far from major population centers. Remote, high-altitude environments provide useful settings for investigating these patterns because atmospheric transport and deposition can carry contaminants to mountain snow, precipitation, and surface waters.

Detecting a chemical signal in an environmental sample is not the same as identifying the compound that produced it. Liquid chromatography–high-resolution mass spectrometry (LC-HRMS) can generate complex datasets containing many molecular features. Some features can be associated with known compounds, while others remain unidentified because available databases and reference spectra do not provide enough evidence for a confident assignment. Researchers need ways to organize and investigate these features without overstating what the data demonstrate.

Molecular networking offers one way to explore these datasets by connecting features whose tandem mass spectra share similarities. A network can help researchers inspect relationships among features and identify groups that warrant further chemical investigation. Its usefulness depends on a workflow that handles spectral data consistently, makes the basis of network connections inspectable, and supports reproducible analysis. Spectral similarity alone does not prove that two features are the same compound or that one compound transforms into another.

This project combines environmental chemistry and software development to investigate how molecular networking can be made reliable, reproducible, and useful for environmental HRMS research at the College of

Wooster. The goal is not to replace chemical confirmation or automatically identify every unknown feature. Instead, the project aims to develop and evaluate a computational workflow and interface that help chemists construct, inspect, and interpret molecular networks while preserving the evidence and limitations associated with each feature.

1.2 Environmental and Chemical Background

1.2.1 Long-Range Transport of Organic Contaminants

Organic contaminants can move away from their original sources through atmospheric processes. Their movement and deposition depend on properties such as volatility and persistence, as well as environmental conditions. Some semivolatile compounds can undergo repeated cycles of volatilization, atmospheric transport, and deposition, allowing them to reach regions distant from where they were emitted. Temperature gradients and precipitation can influence where these chemicals are deposited.

1.2.2 High-Altitude and Remote Surface-Water Environments

Remote, high-altitude environments provide useful settings for studying the distribution of environmental contaminants. Locations in the project's research context include Mount Everest, Mount Kilimanjaro, the Qinghai–Tibetan Plateau, and associated surface-water environments. Snow, precipitation, and surface water can provide evidence of atmospheric transport and deposition. Remoteness does not guarantee isolation from human-associated chemicals.

This project uses existing environmental HRMS data; sample collection is not itself a contribution of this thesis. The origins, sample types, and analytical details of the datasets used in the software evaluation will be documented in Chapter 2.

1.2.3 Previous Wooster Research and the Identification Gap

Previous research associated with the College of Wooster has investigated organic contaminants in remote and high-altitude environmental samples. This work provides the chemical and analytical context for the current project. Even when LC-HRMS detects features of interest, not every feature can be assigned confidently to a specific compound. A feature may have accurate-mass, retention-time, isotope, or MS/MS information without a confirmed chemical identity.

The gap is both chemical and computational. Chemically, researchers need ways to prioritize and interpret features that lack confident identifications. Computationally, they need tools that allow spectral relationships

and associated metadata to be examined consistently. This thesis addresses the computational part of that problem while using environmental data to demonstrate how the workflow can support chemical investigation.

1.3 LC-HRMS, Suspect Screening, and Non-Targeted Analysis

1.3.1 LC-HRMS and Tandem Mass Spectrometry

Liquid chromatography separates components of a sample before they enter a mass spectrometer. High-resolution mass spectrometry measures ions with high mass accuracy and provides information used to characterize molecular features. In tandem mass spectrometry (MS/MS), selected precursor ions are fragmented and the resulting product-ion spectra provide additional evidence about molecular structure. These data support chemical annotation and comparison, but their interpretation depends on data quality and available reference information.

1.3.2 Molecular Features Versus Identified Compounds

A molecular feature is an observed signal characterized by analytical attributes such as mass-to-charge ratio, retention time, and, when available, isotope or MS/MS information. A feature should not automatically be treated as a confirmed compound. Adduct formation, in-source fragmentation, isomers, incomplete spectra, and limited reference information can complicate interpretation.

This thesis uses the term *feature* for detected signals whose chemical identities have not been confirmed. The term *compound* is reserved for cases where the evidence supports that terminology. Network connections and candidate annotations are evidence to investigate, not substitutes for appropriate chemical confirmation.

1.3.3 Suspect Screening and Non-Targeted Analysis

Targeted analysis focuses on a predefined set of compounds. Suspect screening examines data for compounds on a suspect list, including compounds that may not be part of a conventional targeted method. Non-targeted analysis (NTA) aims to detect and investigate a broader range of chemical features without restricting the analysis to a fixed list of target compounds.

These approaches are complementary. Suspect screening can investigate compounds for which there is a prior reason to look, while NTA can retain information about features not anticipated in advance. Neither approach guarantees that every detected feature can be identified. Candidate assignments must be evaluated against available mass, retention-time, isotope, and fragmentation evidence, as applicable.

1.3.4 Limitations of Current Identification Approaches

Reference databases and spectral libraries are incomplete. A detected feature may not have a suitable reference spectrum, and different structures can sometimes produce similar or partially overlapping fragmentation patterns. Data quality, acquisition conditions, and processing choices can also affect the evidence available for interpretation.

Identifying and prioritizing unknown features therefore requires researchers to combine different types of evidence and preserve uncertainty in their conclusions. A computational tool can help organize data and make relationships easier to inspect, but it should not convert a similarity score or network connection into an unsupported claim of chemical identity.

1.4 Molecular Networking and Theoretical Foundations

1.4.1 MS/MS Fragmentation and Spectral Similarity

MS/MS spectra represent product ions generated when a precursor ion fragments under specified conditions. Related molecules may share some fragments or fragmentation patterns, so comparing spectra can provide clues about structural similarities. The strength of these inferences depends on spectrum quality, the comparison method, and the chemical context.

Spectral similarity is best treated as a way to prioritize and explore possible relationships. It is not, on its own, definitive evidence that two features represent the same molecule or lie on a specific transformation pathway.

1.4.2 Cosine Similarity and Molecular-Network Construction

A spectral similarity score quantifies the correspondence between two spectra under a specified comparison procedure. Cosine similarity is one common approach. After spectra are represented and peaks are matched according to defined rules, the score summarizes alignment between the resulting spectral vectors. The score depends on implementation choices such as peak matching, mass tolerances, preprocessing, and filtering.

In a molecular network, each node represents a molecular feature and an edge represents a relationship that meets the selected similarity criteria. Edge weights or labels can retain the similarity score so researchers can inspect the computational relationship. Thresholds affect the resulting graph: stricter criteria may remove weaker connections, while more permissive criteria may create a denser network. The chosen settings and their rationale should therefore be documented.

1.4.3 Graph Theory: Nodes, Edges, and Connected Components

Graph theory provides a formal way to represent relationships among objects. In molecular networking, nodes represent features and edges represent the spectral relationships retained by the workflow. A connected component is a set of nodes connected through paths in the graph. Isolated nodes have no retained edges under the selected network criteria.

These structures help summarize and navigate relationships in a dataset. However, a connected component should not automatically be interpreted as one chemical family, and an edge should not be treated as proof of shared identity or a reaction pathway. Interpretation requires considering the graph alongside spectral evidence, metadata, annotations, and analytical limitations.

1.4.4 Network Visualization and Interpretation

Network visualization can help researchers inspect complex relationships that may be difficult to explore in a table alone. Selecting nodes and edges, viewing feature metadata, examining similarity values, and filtering the network can connect the overall graph to the underlying chemical evidence. Visual encodings should communicate meaningful properties without implying more certainty than the data support.

The interface is most useful when users can move between the network-level view and the evidence behind a connection. Users should be able to inspect which features are connected, why an edge is present under the selected criteria, and what information is available about the features. Important settings and data provenance should be accessible enough to support repeatable analysis.

1.4.5 Limitations of Spectral Similarity as Evidence of Chemical Identity

A high spectral similarity score indicates agreement under the selected computational comparison; it does not independently establish molecular identity. Similar spectra can arise for different structures, and structurally related compounds can produce different spectra under different conditions. Missing or low-quality MS/MS data may also weaken or prevent meaningful comparisons.

The software developed in this project is intended to support exploration of evidence rather than replace expert chemical judgment or confirmatory experiments. Where reference standards or other appropriate confirmation are absent, the thesis will distinguish tentative annotations and possible relationships from confirmed identifications.

1.5 Existing Software and Research Gap

1.5.1 MS-DIAL and GNPS in the Existing Workflow

The existing research workflow uses MS-DIAL for upstream processing of mass-spectrometry data and molecular-networking tools such as the Global Natural Products Social Molecular Networking platform (GNPS) to explore spectral relationships. These tools address different stages of the workflow: processing software prepares and organizes analytical data, while molecular-networking software represents relationships among features.

This project builds on that workflow rather than assuming that any one tool is universally unsuitable. The relevant question is whether the workflow used for the College of Wooster's environmental HRMS research meets practical requirements for data handling, network inspection, reproducibility, and interpretation.

1.5.2 Documented Limitations in the Wooster Research Workflow

The motivation for this project includes workflow difficulties reported during local molecular-networking work, particularly around preserving work and reliably saving or exporting results. These concerns should be described as observed or reported issues in this specific research context, not as a general claim that GNPS or other existing platforms fail for all users.

The project will translate relevant research needs into explicit software requirements. These may include dependable input handling, inspectable node and edge information, links between network elements and chemical metadata, and clear behavior when saving or exporting results. The final requirements should reflect documented needs and features actually implemented and evaluated.

1.5.3 The Need for Software Designed Around Chemists' Needs

Molecular-networking software must do more than generate a graph. It should help users move between a network-level view and the underlying evidence, understand how connections were calculated, and preserve the information needed to reproduce an analysis. The interface should make important details accessible without requiring users to reconstruct the workflow from separate files or undocumented steps.

This project treats software reliability and usability as research concerns alongside the chemical application. The proposed contribution is a workflow and interface tailored to the current environmental HRMS research context, evaluated through computational tests and case studies using real data. Its success will be assessed based on implemented functionality and measured results rather than assumed in advance.

1.6 Research Question and Specific Aims

The central research question is:

> What computational workflow and interface are needed to make molecular networking reliable, reproducible, and useful for Wooster's environmental HRMS research?

The project addresses this question through four aims:

1. **Define the chemistry and data requirements.** Identify the analytical data, metadata, and evidence needed to construct and interpret molecular networks for the environmental HRMS research context.
2. **Develop an end-to-end molecular-networking workflow.** Implement a workflow that processes supported input data, compares MS/MS spectra, constructs a network, and allows users to inspect network elements and associated chemical information.
3. **Evaluate computational correctness, reproducibility, performance, and workflow usefulness.** Test calculations, input handling, graph construction, repeatability, performance, and relevant user-facing tasks using documented evaluation methods.
4. **Apply the software to environmental HRMS case studies.** Use selected environmental data to examine annotated and unidentified features, describe relationships supported by the evidence, and identify remaining uncertainties.

1.7 Project Scope and Thesis Contributions

1.7.1 Project Scope

This project focuses on computational support for molecular networking using existing environmental HRMS data. It includes defining data requirements, implementing and testing the network workflow, designing an interface for inspecting features and spectral relationships, and applying the software to selected chemical case studies. The specific input formats, data-processing assumptions, software features, and evaluation procedures will be defined in Chapter 2.

The project does not assume that a network alone can confirm the identity of unknown compounds, prove chemical transformation pathways, or replace reference standards and other confirmatory methods. Software development alone also does not establish the environmental source, toxicity, or ecological effect of a detected feature. Claims in the chemical case studies will be limited to what the available analytical data and supporting evidence justify.

1.7.2 Expected Computer Science Contributions

The Computer Science contribution is the design, implementation, and evaluation of a molecular-networking workflow and interface for the stated research context. The contribution will be documented through the software architecture, similarity and graph-construction methods, interface behavior, input validation, testing, and reproducibility practices. Chapter 3 will report what was implemented and how it performed under the evaluation procedures.

1.7.3 Expected Chemistry Contributions

The Chemistry contribution is the application of the workflow to environmental HRMS data to investigate spectral relationships among annotated and unidentified molecular features. The case studies will describe the evidence available for selected features, explain how network relationships may help prioritize further investigation, and state the remaining limitations. Tentative network relationships will not be presented as confirmed chemical identifications.

1.7.4 Organization of the Thesis

Chapter 2 describes the research design, environmental datasets and analytical methods, upstream processing, software requirements and implementation, network construction, and evaluation procedures. Chapter 3 presents the dataset summary, software implementation results, computational validation, performance and workflow evaluation, network characteristics, and chemical case studies. Chapter 4 interprets the computational and chemical findings, discusses design trade-offs and limitations, and considers implications for future environmental HRMS research. Chapter 5 summarizes the contributions, answers the central research question, and identifies future directions.

CHAPTER 2

Methodology, Research Design, and Software Development

2.1 Overall Research Design and Workflow

2.2 Environmental Samples and Chemistry Experimental Methods

2.3 Upstream Data Processing and Chemical Annotation

2.4 Software Requirements and System Architecture

2.5 Spectral Comparison and Similarity Calculation

2.6 Molecular-Network Construction

2.7 User Interface Design and Interaction

2.8 Software Implementation and Reproducibility

2.9 Software Evaluation and Testing Methods

2.10 Chemical Case-Study Design

CHAPTER 3

Results and Data Analysis

3.1 Environmental Dataset and Feature Summary

3.2 Software Implementation Results

3.3 Computational Correctness and Validation

3.4 Software Performance and Workflow Evaluation

3.5 Molecular-Network Structure and Characteristics

3.6 Chemical Case Study 1: Annotated Molecular Features

3.7 Chemical Case Study 2: Unidentified Molecular Features

3.8 Summary of Principal Results

CHAPTER 4

Discussion

4.1 Interpretation and Integration of Results

4.2 Computational Contributions and Design Trade-Offs

4.3 Chemical Significance and Comparison with Literature

4.4 Study and Implementation Limitations

4.5 Implications for Future Environmental HRMS Research

CHAPTER 5

Conclusions and Future Work

5.1 Summary of the Research

5.2 Principal Computer Science Contributions

5.3 Principal Chemistry Contributions

5.4 Answer to the Central Research Question

5.5 Future Work

5.6 Final Remarks

APPENDIX A

Typesetting Mathematical Formulae

This appendix is taken from **ophs03** under the GNU open-source documentation license. This appendix addresses the main strength of TEX: mathematical typesetting. But be warned, this appendix only scratches the surface. While the things explained here are sufficient for many people, don't despair if you can't find a solution to your mathematical typesetting needs here. It is highly likely that your problem is addressed in $\mathcal{AMS}$-LATEX[1] or some other package.

A.1 General

LATEX has a special mode for typesetting mathematics. Mathematical text within a paragraph is entered between `\(` and `\)`, between `$` and `$` or between `\begin{math}` and `\end{math}`.

```
Add $a$ squared and $b$ squared
to get $c$ squared. Or, using
a more mathematical approach:
$c^{2}=a^{2}+b^{2}$
```

Add a squared and b squared to get c squared. Or, using a more mathematical approach: $c^2 = a^2 + b^2$

```
\TeX{} is pronounced as
$\tau\epsilon$.\\[6pt]
100~m$^{3}$ of water\\[6pt]
This comes from my $\heartsuit$
```

TEX is pronounced as $\tau\epsilon$.

100 m^3 of water

This comes from my $\heartsuit$

It is preferable to *display* larger mathematical equations or formulae, rather than to typeset them on separate lines. This means you enclose them in `\[` and `\]` or between `\begin{displaymath}` and `\end{displaymath}`. This produces formulae which are not numbered. If you want LATEX to number them, you can use the equation environment.

[1] `CTAN:/tex-archive/macros/latex/packages/amslatex`

```
Add $a$ squared and $b$ squared
to get $c$ squared. Or, using
a more mathematical approach:
\begin{displaymath}
c^{2}=a^{2}+b^{2}
\end{displaymath}
And just one more line.
```

Add a squared and b squared to get c squared. Or, using a more mathematical approach:

$$c^2 = a^2 + b^2$$

And just one more line.

You can reference an equation with \label and \ref

```
\begin{equation} \label{eq:eps}
\epsilon > 0
\end{equation}
From (\ref{eq:eps}), we gather
\ldots
```

$$\epsilon > 0 \tag{A.1}$$

From (A.1), we gather ...

Note that expressions will be typeset in a different style if displayed:

```
$\lim_{n \to \infty}
\sum_{k=1}^n \frac{1}{k^2}
= \frac{\pi^2}{6}$
```

$\lim_{n \to \infty} \sum_{k=1}^{n} \frac{1}{k^2} = \frac{\pi^2}{6}$

```
\begin{displaymath}
\lim_{n \to \infty}
\sum_{k=1}^n \frac{1}{k^2}
= \frac{\pi^2}{6}
\end{displaymath}
```

$$\lim_{n \to \infty} \sum_{k=1}^{n} \frac{1}{k^2} = \frac{\pi^2}{6}$$

There are differences between *math mode* and *text mode*. For example, in *math mode*:

1. Most spaces and linebreaks do not have any significance, as all spaces either are derived logically from the mathematical expressions or have to be specified using special commands such as \, , \quad, or \qquad.
2. Empty lines are not allowed. Only one paragraph per formula.
3. Each letter is considered to be the name of a variable and will be typeset as such. If you want to typeset normal text within a formula (normal upright font and normal spacing) then you have to enter the text using the \textrm{...} commands.

```
\begin{equation}
\forall x \in \mathbf{R}:
\qquad x^{2} \geq 0
\end{equation}
```

$$\forall x \in \mathbf{R}: \qquad x^2 \geq 0 \tag{A.2}$$

```
\begin{equation}
x^{2} \geq 0\qquad
\textrm{for all }x\in\mathbf{R}
\end{equation}
```

$$x^2 \geq 0 \qquad \text{for all } x \in \mathbf{R} \tag{A.3}$$

Mathematicians can be very fussy about which symbols are used: it would be conventional here to use 'blackboard bold', bold symbols which is obtained using \mathbb from the package amsfonts or amssymb.

The last example becomes

```
\begin{displaymath}
x^{2} \geq 0\qquad
\textrm{for all }x\in\mathbb{R}
\end{displaymath}
```

$$x^2 \geq 0 \qquad \text{for all } x \in \mathbb{R}$$

A.2 Grouping in Math Mode

Most math mode commands act only on the next character. So if you want a command to affect several characters, you have to group them together using curly braces: {...}.

```
\begin{equation}
a^x+y \neq a^{x+y}
\end{equation}
```

$$a^x + y \neq a^{x+y} \tag{A.4}$$

A.3 Building Blocks of a Mathematical Formula

In this section, the most important commands used in mathematical typesetting will be described. Take a look at **kd03** for a detailed list of commands for typesetting mathematical symbols.

Lowercase Greek letters are entered as \alpha, \beta, \gamma, ..., uppercase letters are entered as \Gamma, \Delta, ...[2]

```
$\lambda,\xi,\pi,\mu,\Phi,\Omega$
```

$\lambda, \xi, \pi, \mu, \Phi, \Omega$

Exponents and Subscripts can be specified using the ^ and the _ character.

```
$a_{1}$ \qquad $x^{2}$ \qquad
$e^{-\alpha t}$ \qquad
$a^{3}_{ij}$\\
$e^{x^2} \neq {e^x}^2$
```

$a_1 \qquad x^2 \qquad e^{-\alpha t} \qquad a^3_{ij}$
$e^{x^2} \neq e^{x^2}$

[2] There is no uppercase Alpha defined in LaTeX 2_ε because it looks the same as a normal roman A. Once the new math coding is done, things will change.

The **square root** is entered as \sqrt, the nth root is generated with \sqrt[n]. The size of the root sign is determined automatically by LaTeX. If just the sign is needed, use \surd.

```
$\sqrt{x}$ \qquad
$\sqrt{ x^{2}+\sqrt{y} }$
\qquad $\sqrt[3]{2}$\\[3pt]
$\surd[x^2 + y^2]$
```

$\sqrt{x}$ $\sqrt{ x^{2}+\sqrt{y} }$ $\sqrt[3]{2}$
$\surd[x^2 + y^2]$

The commands \overline and \underline create **horizontal lines** directly over or under an expression.

```
$\overline{m+n}$
```

$\overline{m+n}$

The commands \overbrace and \underbrace create long **horizontal braces** over or under an expression.

```
$\underbrace{ a+b+\cdots+z }_{26}$
```

$\underbrace{ a+b+\cdots+z }_{26}$

To add mathematical accents such as small arrows or tilde signs to variables, you can use the commands given in **kd03**. Wide hats and tildes covering several characters are generated with \widetilde and \widehat. The ' symbol gives a prime.

```
\begin{displaymath}
y=x^{2}\qquad y'=2x\qquad y''=2
\end{displaymath}
```

$$y=x^{2} \qquad y'=2x \qquad y''=2$$

Vectors often are specified by adding a small arrow symbol on top of a variable. This is done with the \vec command. The two commands \overrightarrow and \overleftarrow are useful to denote the vector from A to B.

```
\begin{displaymath}
\vec a\quad\overrightarrow{AB}
\end{displaymath}
```

$$\vec{a} \quad \overrightarrow{AB}$$

Names of log-like functions are often typeset in an upright font and not in italic like variables. Therefore LaTeX supplies the following commands to typeset the most important function names:

```
\arccos   \cos    \csc   \exp   \ker     \limsup  \min   \sinh
\arcsin   \cosh   \deg   \gcd   \lg      \ln      \Pr    \sup
\arctan   \cot    \det   \hom   \lim     \log     \sec   \tan
\arg      \coth   \dim   \inf   \liminf  \max     \sin   \tanh
```

```
\[\lim_{x \rightarrow 0}
\frac{\sin x}{x}=1\]
```

$$\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$$

For the modulo function, there are two commands: `\bmod` for the binary operator "$a \bmod b$" and `\pmod` for expressions such as "$x \equiv a \pmod{b}$."

A built-up **fraction** is typeset with the `\frac{...}{...}` command. Often the slashed form `1/2` is preferable, because it looks better for small amounts of 'fraction material.'

```
$1\frac{1}{2}$~hours
\begin{displaymath}
\frac{ x^{2} }{ k+1 }\qquad
x^{ \frac{2}{k+1} }\qquad
x^{ 1/2 }
\end{displaymath}
```

$1\frac{1}{2}$ hours

$$\frac{x^2}{k+1} \qquad x^{\frac{2}{k+1}} \qquad x^{1/2}$$

To typeset binomial coefficients or similar structures, you can use either the command `\binom{`*num*`}{`*denom*`}` or `\genfrac{`*ldelim*`}{`*rdelim*`}{`*thickness*`}{`*style*`}{`*num*`}{`*denom*`}`. The second command can be used to produce customized fraction like output and more information can be found in **mgbcr04**.

```
\begin{displaymath}
\binom{n}{k}\qquad
\genfrac{}{}{0pt}{}{x}{y+2}
\end{displaymath}
```

$$\binom{n}{k} \qquad \genfrac{}{}{0pt}{}{x}{y+2}$$

The **integral operator** is generated with `\int`, the **sum operator** with `\sum`. The upper and lower limits are specified with `^` and `_` like subscripts and superscripts.

```
\begin{displaymath}
\sum_{i=1}^{n} \qquad
\int_{0}^{\frac{\pi}{2}} \qquad
\end{displaymath}
```

$$\sum_{i=1}^{n} \qquad \int_{0}^{\frac{\pi}{2}}$$

For **braces** and other delimiters, there exist all types of symbols in TEX (e.g. $[\ \langle\ \|\ \updownarrow$). Round and square braces can be entered with the corresponding keys, curly braces with `\{`, all other delimiters are generated with special commands (e.g. `\updownarrow`). For a list of all delimiters available, check **kd03**.

```
\begin{displaymath}
{a,b,c}\neq\{a,b,c\}
\end{displaymath}
```

$$a, b, c \neq \{a, b, c\}$$

If you put the command \left in front of an opening delimiter or \right in front of a closing delimiter, TEX will automatically determine the correct size of the delimiter. Note that you must close every \left with a corresponding \right, and that the size is determined correctly only if both are typeset on the same line. If you don't want anything on the right, use the invisible '\right .'!

```
\begin{displaymath}
1 + \left( \frac{1}{ 1-x^{2} }
    \right) ^3
\end{displaymath}
```

$$1 + \left(\frac{1}{ 1-x^{2} } \right) ^3$$

In some cases it is necessary to specify the correct size of a mathematical delimiter by hand, which can be done using the commands \big, \Big, \bigg and \Bigg as prefixes to most delimiter commands.[3]

```
$\Big( (x+1) (x-1) \Big) ^{2}$\\
$\big(\Big(\bigg(\Bigg($\quad
$\big\}\Big\}\bigg\}\Bigg\}$\quad
$\big\|\Big\|\bigg\|\Bigg\|$
```

$$\Big((x+1) (x-1) \Big) ^{2}$$
$$\big(\Big(\bigg(\Bigg(\quad \big\}\Big\}\bigg\}\Bigg\} \quad \big\|\Big\|\bigg\|\Bigg\|$$

To enter **three dots** into a formula, you can use several commands. \ldots typesets the dots on the baseline, \cdots sets them centered. Besides that, there are the commands \vdots for vertical and \ddots for diagonal dots. You can find another example in section A.5.

```
\begin{displaymath}
x_{1},\ldots,x_{n} \qquad
x_{1}+\cdots+x_{n}
\end{displaymath}
```

$$x_{1},\ldots,x_{n} \qquad x_{1}+\cdots+x_{n}$$

A.4 Math Spacing

If the spaces within formulae chosen by TEX are not satisfactory, they can be adjusted by inserting special spacing commands. There are some commands for small spaces: \, for $\frac{3}{18}$ quad (␣), \: for $\frac{4}{18}$ quad (␣) and \; for $\frac{5}{18}$ quad (␣). The escaped space character \␣ generates a medium sized space and \quad (␣␣) and \qquad (␣␣␣␣) produce large spaces. The size of a quad corresponds to the width of the character 'M' of the current font. The \! command produces a negative space of $-\frac{3}{18}$ quad (␣).

[3] These commands do not work as expected if a size changing command has been used, or the 11pt or 12pt option has been specified. Use the exscale or amsmath packages to correct this behaviour.

```
\newcommand{\rd}{\mathrm{d}}
\begin{displaymath}
\int\!\!\!\int_{D} g(x,y)
  \, \rd x\, \rd y
\end{displaymath}
instead of
\begin{displaymath}
\int\int_{D} g(x,y)\rd x \rd y
\end{displaymath}
```

$$\iint_D g(x,y)\,\mathrm{d}x\,\mathrm{d}y$$

instead of

$$\int\int_D g(x,y)\mathrm{d}x\mathrm{d}y$$

Note that 'd' in the differential is conventionally set in roman.

$\mathcal{AMS}$-LaTeX provides another way for fine tuning the spacing between multiple integral signs, namely the `\iint`, `\iiint`, `\iiiint`, and `\idotsint` commands. With the `amsmath` package loaded, the above example can be typeset this way:

```
\newcommand{\rd}{\mathrm{d}}
\begin{displaymath}
\iint_{D} \, \rd x \, \rd y
\end{displaymath}
```

$$\iint_D \mathrm{d}x\,\mathrm{d}y$$

See the electronic document testmath.tex (distributed with $\mathcal{AMS}$-LaTeX) or Chapter 8 of "The LaTeX Companion"[4] for further details.

A.5 Vertically Aligned Material

To typeset **arrays**, use the `array` environment. It works somewhat similar to the `tabular` environment. The `\\` command is used to break the lines.

```
\begin{displaymath}
\mathbf{X} =
\left( \begin{array}{ccc}
x_{11} & x_{12} & \ldots \\
x_{21} & x_{22} & \ldots \\
\vdots & \vdots & \ddots
\end{array} \right)
\end{displaymath}
```

$$\mathbf{X} = \left(\begin{array}{ccc} x_{11} & x_{12} & \ldots \\ x_{21} & x_{22} & \ldots \\ \vdots & \vdots & \ddots \end{array} \right)$$

The `array` environment can also be used to typeset expressions which have one big delimiter by using a "`.`" as an invisible right delimiter:

[4] available at `CTAN:/tex-archive/info/ch8.*`.

```
\begin{displaymath}
y = \left\{ \begin{array}{ll}
 a & \textrm{if $d>c$}\\
 b+x & \textrm{in the morning}\\
 l & \textrm{all day long}
  \end{array} \right.
\end{displaymath}
```

$$y = \begin{cases} a & \text{if } d > c \\ b + x & \text{in the morning} \\ l & \text{all day long} \end{cases}$$

For formulae running over several lines or for equation systems, you can use the environments `eqnarray`, and `eqnarray*` instead of `equation`. In `eqnarray` each line gets an equation number. The `eqnarray*` does not number anything.

The `eqnarray` and the `eqnarray*` environments work like a 3-column table of the form `{rcl}`, where the middle column can be used for the equal sign or the not-equal sign. Or any other sign you see fit. The `\\` command breaks the lines.

```
\begin{eqnarray}
f(x) & = & \cos x     \\
f'(x) & = & -\sin x   \\
\int_{0}^{x} f(y)dy &
 = & \sin x
\end{eqnarray}
```

$$\begin{aligned} f(x) &= \cos x && \text{(A.5)} \\ f'(x) &= -\sin x && \text{(A.6)} \\ \int_0^x f(y)dy &= \sin x && \text{(A.7)} \end{aligned}$$

Notice that the space on either side of the the equal signs is rather large. It can be reduced by setting `\setlength\arraycolsep{2pt}`, as in the next example.

Long equations will not be automatically divided into neat bits. The author has to specify where to break them and how much to indent. The following two methods are the most common ones used to achieve this.

```
{\setlength\arraycolsep{2pt}
\begin{eqnarray}\notag
\sin x & = & x -\frac{x^{3}}{3!}
     +\frac{x^{5}}{5!}-{}
                    \\\notag
 & & {}-\frac{x^{7}}{7!}+{}\cdots
\end{eqnarray}}
```

$$\begin{aligned} \sin x \;=\; & x - \frac{x^3}{3!} + \frac{x^5}{5!} - \\ & - \frac{x^7}{7!} + \cdots \end{aligned}$$

```
\begin{eqnarray}\notag
\lefteqn{ \cos x = 1
     -\frac{x^{2}}{2!} +{} }
                    \\\notag
 & & {}+\frac{x^{4}}{4!}
     -\frac{x^{6}}{6!}+{}\cdots
\end{eqnarray}
```

$$\begin{aligned} \cos x = 1 - \frac{x^2}{2!} + \\ + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots \end{aligned}$$

The `\notag` command causes LaTeX to not generate a number for this equation.

It can be difficult to get vertically aligned equations to look right with these methods; the package amsmath provides a more powerful set of alternatives.

A.6 Math Font Size

In math mode, TEX selects the font size according to the context. Superscripts, for example, get typeset in a smaller font. If you want to typeset part of an equation in roman, don't use the `\textrm` command, because the font size switching mechanism will not work, as `\textrm` temporarily escapes to text mode. Use `\mathrm` instead to keep the size switching mechanism active. But pay attention, `\mathrm` will only work well on short items. Spaces are still not active and accented characters do not work.[5]

```
\begin{equation}
2^{\textrm{nd}} \quad
2^{\mathrm{nd}}
\end{equation}
```

$$2^{\textrm{nd}} \quad 2^{\mathrm{nd}} \tag{A.8}$$

Nevertheless, sometimes you need to tell LATEX the correct font size. In math mode, the font size is set with the four commands:

displaystyle (123), textstyle (123), scriptstyle (123) and scriptscriptstyle (123).

Changing styles also affects the way limits are displayed.

```
\begin{displaymath}
\mathop{\mathrm{corr}}(X,Y)=
 \frac{\displaystyle
   \sum_{i=1}^n(x_i-\overline x)
   (y_i-\overline y)}
  {\displaystyle\biggl[
 \sum_{i=1}^n(x_i-\overline x)^2
\sum_{i=1}^n(y_i-\overline y)^2
\biggr]^{1/2}}
\end{displaymath}
```

$$\operatorname{corr}(X,Y) = \frac{\displaystyle \sum_{i=1}^{n}(x_i-\overline{x})(y_i-\overline{y})}{\displaystyle \biggl[\sum_{i=1}^{n}(x_i-\overline{x})^2 \sum_{i=1}^{n}(y_i-\overline{y})^2\biggr]^{1/2}}$$

This is one of those examples in which we need larger brackets than the standard `\left[ \right]` provides.

[5] The $\mathcal{AMS}$-LATEX package makes the textrm command work with size changing.

A.7 Theorems, Laws, ...

When writing mathematical documents, you probably need a way to typeset "Lemmas", "Definitions", "Axioms" and similar structures. LaTeX supports this with the command

newtheorem{*name*}[*counter*]{*text*}[*section*]

The *name* argument, is a short keyword used to identify the "theorem". With the *text* argument, you define the actual name of the "theorem" which will be printed in the final document.

The arguments in square brackets are optional. They are both used to specify the numbering used on the "theorem". With the *counter* argument you can specify the *name* of a previously declared "theorem". The new "theorem" will then be numbered in the same sequence. The *section* argument allows you to specify the sectional unit within which you want your "theorem" to be numbered.

After executing the newtheorem command in the preamble of your document, you can use the following command within the document.

\begin{*name*}[*text*] This is my interesting theorem \end{*name*}

This should be enough theory. The following examples will hopefully remove the final remains of doubt and make it clear that the \newtheorem environment is way too complex to understand.

```
% definitions for the document
% preamble
\newtheorem{law}{Law}
\newtheorem{jury}[law]{Jury}
%in the document
\begin{law} \label{law:box}
Don't hide in the witness box
\end{law}
\begin{jury}[The Twelve]
It could be you! So beware and
see law~\ref{law:box}\end{jury}
\begin{law}No, No, No\end{law}
```

Law 1. *Don't hide in the witness box*

Jury 2 (The Twelve)**.** *It could be you! So beware and see law 1*

Law 3. *No, No, No*

The "Jury" theorem uses the same counter as the "Law" theorem. Therefore it gets a number which is in sequence with the other "Laws". The argument in square brackets is used to specify a title or something similar for the theorem.

```
\flushleft
\newtheorem{mur}{Murphy}[section]
\begin{mur}
If there are two or more
ways to do something, and
one of those ways can result
in a catastrophe, then
someone will do it.\end{mur}
```

Murphy A.7.1. *If there are two or more ways to do something, and one of those ways can result in a catastrophe, then someone will do it.*

The "Murphy" theorem gets a number which is linked to the number of the current section. You could also use another unit, for example chapter or subsection.

A.8 Bold symbols

It is quite difficult to get bold symbols in LaTeX; this is probably intentional as amateur typesetters tend to overuse them. The font change command `\mathbf` gives bold letters, but these are roman (upright) whereas mathematical symbols are normally italic. There is a `\boldmath` command, but *this can only be used outside mathematics mode*. It works for symbols too.

```
\begin{displaymath}\label{boldmath}
\mu, M \qquad \mathbf{M} \qquad
\mbox{\boldmath $\mu, M$}
\end{displaymath}
```

$$\mu, M \qquad \mathbf{M} \qquad \boldsymbol{\mu, M}$$

Notice that the comma is bold too, which may not be what is required.

The package `amsbsy` (included by `amsmath`) makes this much easier as it includes a `\boldsymbol` command.

```
\begin{displaymath}\label{boldsymbol}
\mu, M \qquad
\boldsymbol{\mu}, \boldsymbol{M}
\end{displaymath}
```

$$\mu, M \qquad \boldsymbol{\mu}, \boldsymbol{M}$$

A.9 List of Mathematical Symbols

In the following tables, you find all the symbols normally accessible from *math mode*.

To use the symbols listed in Tables A.12–A.17,[6] the package `amssymb` must be loaded in the preamble of the document and the AMS math fonts must be installed, on the system. If the AMS package and fonts are not installed, on your system, have a look at `CTAN:/tex-archive/macros/latex/required/amslatex`

[6]These tables were derived from `symbols.tex` by David Carlisle and subsequently changed extensively as suggested by Josef Tkadlec.

Table A.1: Math Mode Accents.

$\hat{a}$	`\hat{a}`	$\check{a}$	`\check{a}`	$\tilde{a}$	`\tilde{a}`	$\acute{a}$	`\acute{a}`
$\grave{a}$	`\grave{a}`	$\dot{a}$	`\dot{a}`	$\ddot{a}$	`\ddot{a}`	$\breve{a}$	`\breve{a}`
$\bar{a}$	`\bar{a}`	$\vec{a}$	`\vec{a}`	$\widehat{A}$	`\widehat{A}`	$\widetilde{A}$	`\widetilde{A}`

Table A.2: Lowercase Greek Letters.

α	`\alpha`	θ	`\theta`	o	`o`	υ	`\upsilon`
β	`\beta`	ϑ	`\vartheta`	π	`\pi`	ϕ	`\phi`
γ	`\gamma`	ι	`\iota`	ϖ	`\varpi`	φ	`\varphi`
δ	`\delta`	κ	`\kappa`	ρ	`\rho`	χ	`\chi`
ϵ	`\epsilon`	λ	`\lambda`	ϱ	`\varrho`	ψ	`\psi`
ε	`\varepsilon`	μ	`\mu`	σ	`\sigma`	ω	`\omega`
ζ	`\zeta`	ν	`\nu`	ς	`\varsigma`		
η	`\eta`	ξ	`\xi`	τ	`\tau`		

Table A.3: Uppercase Greek Letters.

Γ	`\Gamma`	Λ	`\Lambda`	Σ	`\Sigma`	Ψ	`\Psi`
Δ	`\Delta`	Ξ	`\Xi`	Υ	`\Upsilon`	Ω	`\Omega`
Θ	`\Theta`	Π	`\Pi`	Φ	`\Phi`		

Table A.4: Binary Relations.

You can produce corresponding negations by adding a `\not` command as prefix to the following symbols.

$<$	`<`	$>$	`>`	$=$	`=`
$\leq$	`\leq` or `\le`	$\geq$	`\geq` or `\ge`	$\equiv$	`\equiv`
$\ll$	`\ll`	$\gg$	`\gg`	$\doteq$	`\doteq`
$\prec$	`\prec`	$\succ$	`\succ`	$\sim$	`\sim`
$\preceq$	`\preceq`	$\succeq$	`\succeq`	$\simeq$	`\simeq`
$\subset$	`\subset`	$\supset$	`\supset`	$\approx$	`\approx`
$\subseteq$	`\subseteq`	$\supseteq$	`\supseteq`	$\cong$	`\cong`
$\sqsubset$ [a]	`\sqsubset` [a]	$\sqsupset$ [a]	`\sqsupset` [a]	$\bowtie$	`\Join` [a]
$\sqsubseteq$	`\sqsubseteq`	$\sqsupseteq$	`\sqsupseteq`	$\bowtie$	`\bowtie`
$\in$	`\in`	$\ni$	`\ni` , `\owns`	$\propto$	`\propto`
$\vdash$	`\vdash`	$\dashv$	`\dashv`	$\models$	`\models`
$\mid$	`\mid`	$\parallel$	`\parallel`	$\perp$	`\perp`
$\smile$	`\smile`	$\frown$	`\frown`	$\asymp$	`\asymp`
$:$	`:`	$\notin$	`\notin`	$\neq$	`\neq` or `\ne`

[a] Use the `latexsym` package to access this symbol

Table A.5: Binary Operators.

$+$	`+`	$-$	`-`		
$\pm$	`\pm`	$\mp$	`\mp`	$\triangleleft$	`\triangleleft`
$\cdot$	`\cdot`	$\div$	`\div`	$\triangleright$	`\triangleright`
$\times$	`\times`	$\setminus$	`\setminus`	$\star$	`\star`
$\cup$	`\cup`	$\cap$	`\cap`	$*$	`\ast`
$\sqcup$	`\sqcup`	$\sqcap$	`\sqcap`	$\circ$	`\circ`
$\vee$	`\vee` , `\lor`	$\wedge$	`\wedge` , `\land`	$\bullet$	`\bullet`
$\oplus$	`\oplus`	$\ominus$	`\ominus`	$\diamond$	`\diamond`
$\odot$	`\odot`	$\oslash$	`\oslash`	$\uplus$	`\uplus`
$\otimes$	`\otimes`	$\bigcirc$	`\bigcirc`	$\amalg$	`\amalg`
$\bigtriangleup$	`\bigtriangleup`	$\bigtriangledown$	`\bigtriangledown`	$\dagger$	`\dagger`
$\lhd$ [a]	`\lhd` [a]	$\rhd$ [a]	`\rhd` [a]	$\ddagger$	`\ddagger`
$\unlhd$ [a]	`\unlhd` [a]	$\unrhd$ [a]	`\unrhd` [a]	$\wr$	`\wr`

Table A.6: BIG Operators.

$\sum$	`\sum`	$\bigcup$	`\bigcup`	$\bigvee$	`\bigvee`	$\bigoplus$	`\bigoplus`
$\prod$	`\prod`	$\bigcap$	`\bigcap`	$\bigwedge$	`\bigwedge`	$\bigotimes$	`\bigotimes`
$\coprod$	`\coprod`	$\bigsqcup$	`\bigsqcup`			$\bigodot$	`\bigodot`
$\int$	`\int`	$\oint$	`\oint`			$\biguplus$	`\biguplus`

Table A.7: Arrows.

$\leftarrow$	\leftarrow or \gets	$\longleftarrow$	\longleftarrow	$\uparrow$	\uparrow
$\rightarrow$	\rightarrow or \to	$\longrightarrow$	\longrightarrow	$\downarrow$	\downarrow
$\leftrightarrow$	\leftrightarrow	$\longleftrightarrow$	\longleftrightarrow	$\updownarrow$	\updownarrow
$\Leftarrow$	\Leftarrow	$\Longleftarrow$	\Longleftarrow	$\Uparrow$	\Uparrow
$\Rightarrow$	\Rightarrow	$\Longrightarrow$	\Longrightarrow	$\Downarrow$	\Downarrow
$\Leftrightarrow$	\Leftrightarrow	$\Longleftrightarrow$	\Longleftrightarrow	$\Updownarrow$	\Updownarrow
$\mapsto$	\mapsto	$\longmapsto$	\longmapsto	$\nearrow$	\nearrow
$\hookleftarrow$	\hookleftarrow	$\hookrightarrow$	\hookrightarrow	$\searrow$	\searrow
$\leftharpoonup$	\leftharpoonup	$\rightharpoonup$	\rightharpoonup	$\swarrow$	\swarrow
$\leftharpoondown$	\leftharpoondown	$\rightharpoondown$	\rightharpoondown	$\nwarrow$	\nwarrow
$\rightleftharpoons$	\rightleftharpoons	$\Longleftrightarrow$	\iff (bigger spaces)	$\leadsto$ [a]	\leadsto [a]

[a] Use the `latexsym` package to access this symbol

Table A.8: Delimiters.

(	(	)	)	$\uparrow$	\uparrow	$\Uparrow$	\Uparrow
[	[or \lbrack	]	] or \rbrack	$\downarrow$	\downarrow	$\Downarrow$	\Downarrow
{	\{ or \lbrace	}	\} or \rbrace	$\updownarrow$	\updownarrow	$\Updownarrow$	\Updownarrow
$\langle$	\langle	$\rangle$	\rangle	$\vert$	| or \vert	$\Vert$	\\| or \Vert
$\lfloor$	\lfloor	$\rfloor$	\rfloor	$\lceil$	\lceil	$\rceil$	\rceil
/	/	\	\backslash		. (dual. empty)		

Table A.9: Large Delimiters.

$\lgroup$	\lgroup	$\rgroup$	\rgroup	$\lmoustache$	\lmoustache	$\rmoustache$	\rmoustache
$\arrowvert$	\arrowvert	$\Arrowvert$	\Arrowvert	$\bracevert$	\bracevert		

Table A.10: Miscellaneous Symbols.

...	\dots	$\cdots$	\cdots	$\vdots$	\vdots	$\ddots$	\ddots
$\hbar$	\hbar	$\imath$	\imath	$\jmath$	\jmath	ℓ	\ell
$\Re$	\Re	$\Im$	\Im	$\aleph$	\aleph	$\wp$	\wp
$\forall$	\forall	$\exists$	\exists	$\mho$ [a]	\mho [a]	∂	\partial
$'$	'	$\prime$	\prime	$\emptyset$	\emptyset	∞	\infty
∇	\nabla	$\triangle$	\triangle	$\Box$ [a]	\Box [a]	$\Diamond$ [a]	\Diamond [a]
$\bot$	\bot	$\top$	\top	$\angle$	\angle	$\surd$	\surd
$\diamondsuit$	\diamondsuit	$\heartsuit$	\heartsuit	$\clubsuit$	\clubsuit	$\spadesuit$	\spadesuit
$\neg$	\neg or \lnot	$\flat$	\flat	$\natural$	\natural	$\sharp$	\sharp

[a] Use the `latexsym` package to access this symbol

Table A.11: Non-Mathematical Symbols.

These symbols can also be used in text mode.

†	\dag	§	\S	©	\copyright
‡	\ddag	¶	\P	£	\pounds

Table A.12: AMS Delimiters.

⌜	\ulcorner	⌝	\urcorner	⌞	\llcorner	⌟	\lrcorner

Table A.13: AMS Greek and Hebrew.

F	\digamma	$\varkappa$	\varkappa	ב	\beth	ד	\daleth	ג	\gimel

Table A.14: AMS Binary Relations.

⩽	\lessdot	⩾	\gtrdot	≒	\doteqdot or \Doteq
⩽	\leqslant	⩾	\geqslant	≓	\risingdotseq
⪕	\eqslantless	⪖	\eqslantgtr	≒	\fallingdotseq
≦	\leqq	≧	\geqq	≖	\eqcirc
⋘	\lll or \llless	⋙	\ggg or \gggtr	≗	\circeq
≲	\lesssim	≳	\gtrsim	≜	\triangleq
≲	\lessapprox	≳	\gtrapprox	≏	\bumpeq
≶	\lessgtr	≷	\gtrless	≎	\Bumpeq
⋚	\lesseqgtr	⋛	\gtreqless	∼	\thicksim
⋚	\lesseqqgtr	⋛	\gtreqqless	≈	\thickapprox
≼	\preccurlyeq	≽	\succcurlyeq	≅	\approxeq

Table A.15: AMS Binary Relations Continued.

⋞	\curlyeqprec	⋟	\curlyeqsucc	∽	\backsim
≾	\precsim	≿	\succsim	≌	\backsimeq
≾	\precapprox	≿	\succapprox	⊨	\vDash
⊆	\subseteqq	⊇	\supseteqq	⊩	\Vdash
⋐	\Subset	⋑	\Supset	⊪	\Vvdash
⊏	\sqsubset	⊐	\sqsupset	∍	\backepsilon
∴	\therefore	∵	\because	∝	\varpropto
∣	\shortmid	∥	\shortparallel	≬	\between
⌣	\smallsmile	⌢	\smallfrown	⋔	\pitchfork
◁	\vartriangleleft	▷	\vartriangleright	◀	\blacktriangleleft
⊴	\trianglelefteq	⊵	\trianglerighteq	▶	\blacktriangleright

Table A.16: AMS Arrows.

⇠	\dashleftarrow	⇢	\dashrightarrow	⊸	\multimap
⇇	\leftleftarrows	⇉	\rightrightarrows	⇈	\upuparrows
⇆	\leftrightarrows	⇄	\rightleftarrows	⇊	\downdownarrows
⇚	\Lleftarrow	⇛	\Rrightarrow	↿	\upharpoonleft
↞	\twoheadleftarrow	↠	\twoheadrightarrow	↾	\upharpoonright
↢	\leftarrowtail	↣	\rightarrowtail	⇃	\downharpoonleft
⇋	\leftrightharpoons	⇌	\rightleftharpoons	⇂	\downharpoonright
↰	\Lsh	↱	\Rsh	⇝	\rightsquigarrow
↫	\looparrowleft	↬	\looparrowright	↭	\leftrightsquigarrow
↶	\curvearrowleft	↷	\curvearrowright		
↺	\circlearrowleft	↻	\circlearrowright		

Table A.17: AMS Negated Binary Relations and Arrows.

≮	\nless	≯	\ngtr	⫋	\varsubsetneqq
≨	\lneq	≩	\gneq	⫌	\varsupsetneqq
≰	\nleq	≱	\ngeq	⫅̸	\nsubseteqq
⩽̸	\nleqslant	⩾̸	\ngeqslant	⫆̸	\nsupseteqq
≨	\lneqq	≩	\gneqq	∤	\nmid
≨	\lvertneqq	≩	\gvertneqq	∦	\nparallel
≰	\nleqq	≱	\ngeqq	∤	\nshortmid
⋦	\lnsim	⋧	\gnsim	∦	\nshortparallel
⪉	\lnapprox	⪊	\gnapprox	≁	\nsim
⊀	\nprec	⊁	\nsucc	≇	\ncong
⪯̸	\npreceq	⪰̸	\nsucceq	⊬	\nvdash
⋨	\precneqq	⋩	\succneqq	⊭	\nvDash
⋨	\precnsim	⋩	\succnsim	⊮	\nVdash
⪉	\precnapprox	⪊	\succnapprox	⊯	\nVDash
⊊	\subsetneq	⊋	\supsetneq	⋪	\ntriangleleft
⫋	\varsubsetneq	⫌	\varsupsetneq	⋫	\ntriangleright
⫅̸	\nsubseteq	⫆̸	\nsupseteq	⋬	\ntrianglelefteq
⫋	\subsetneqq	⫌	\supsetneqq	⋭	\ntrianglerighteq
↚	\nleftarrow	↛	\nrightarrow	↮	\nleftrightarrow
⇍	\nLeftarrow	⇏	\nRightarrow	⇎	\nLeftrightarrow

Table A.18: AMS Binary Operators.

$\dotplus$	`\dotplus`	$\centerdot$	`\centerdot`	$\intercal$	`\intercal`
$\ltimes$	`\ltimes`	$\rtimes$	`\rtimes`	$\divideontimes$	`\divideontimes`
$\Cup$	`\Cup` or `\doublecup`	$\Cap$	`\Cap` or `\doublecap`	$\smallsetminus$	`\smallsetminus`
$\veebar$	`\veebar`	$\barwedge$	`\barwedge`	$\doublebarwedge$	`\doublebarwedge`
$\boxplus$	`\boxplus`	$\boxminus$	`\boxminus`	$\circleddash$	`\circleddash`
$\boxtimes$	`\boxtimes`	$\boxdot$	`\boxdot`	$\circledcirc$	`\circledcirc`
$\leftthreetimes$	`\leftthreetimes`	$\rightthreetimes$	`\rightthreetimes`	$\circledast$	`\circledast`
$\curlyvee$	`\curlyvee`	$\curlywedge$	`\curlywedge`		

Table A.19: AMS Miscellaneous.

$\hbar$	`\hbar`	$\hslash$	`\hslash`	$\Bbbk$	`\Bbbk`
$\square$	`\square`	$\blacksquare$	`\blacksquare`	$\circledS$	`\circledS`
$\vartriangle$	`\vartriangle`	$\blacktriangle$	`\blacktriangle`	$\complement$	`\complement`
$\triangledown$	`\triangledown`	$\blacktriangledown$	`\blacktriangledown`	$\Game$	`\Game`
$\lozenge$	`\lozenge`	$\blacklozenge$	`\blacklozenge`	$\bigstar$	`\bigstar`
$\angle$	`\angle`	$\measuredangle$	`\measuredangle`	$\sphericalangle$	`\sphericalangle`
$\diagup$	`\diagup`	$\diagdown$	`\diagdown`	$\backprime$	`\backprime`
$\nexists$	`\nexists`	$\Finv$	`\Finv`	$\varnothing$	`\varnothing`
$\eth$	`\eth`	$\mho$	`\mho`		

Table A.20: Math Alphabets.

Example	Command	Required package
ABCdef	`\mathrm{ABCdef}`	
$ABCdef$	`\mathit{ABCdef}`	
$ABCdef$	`\mathnormal{ABCdef}`	
$\mathcal{ABC}$	`\mathcal{ABC}`	
$\mathcal{ABC}$	`\mathcal{ABC}`	`eucal` with option: or
	`\mathscr{ABC}`	`eucal` with option: mathscr
$\mathfrak{ABCdef}$	`\mathfrak{ABCdef}`	`eufrak`
$\mathbb{ABC}$	`\mathbb{ABC}`	`amsfonts` or `amssymb`

APPENDIX B

Examples of Java Code

Here are some examples of Java source using the `listings` package. I have entered the following before any code examples to format the code as shown.

```
\lstset{language=java}
\lstset{backgroundcolor=\color{white},rulecolor=\color{black}}
\lstset{linewidth=.95\textwidth,breaklines=true}
\lstset{commentstyle=\textit,stringstyle=\upshape,showspaces=false}
\lstset{frame = trbl, frameround=tttt}
\lstset{numbers=left,numberstyle=\tiny,basicstyle=\small}
\lstset{commentstyle=\normalfont\itshape,breakautoindent=true}
\lstset{abovecaptionskip=1.2\baselineskip,xleftmargin=30pt}
\lstset{framesep=6pt}
```

I have included the code by entering

```
\begin{singlespace}
\lstinputlisting[caption=Clock Code,label=clock]{source/Clock.java}
\end{singlespace}
```

Listing B.1: Clock Code

```
1 //file: Clock.java
2 public class Clock extends UpdateApplet {
3     public void paint( java.awt.Graphics g ) {
4         g.drawString( new java.util.Date().toString( ), 10, 25 );
5     }
6 }
```

Listing B.2: Consumer

```
1  //file: Consumer.java
2  import java.util.Vector;
3
4  public class Consumer implements Runnable
5  {
6      Producer producer;
7
8      Consumer( Producer producer ) {
9          this.producer = producer;
10     }
11
12     public void run() {
13         while ( true ) {
14             String message = producer.getMessage();
15             System.out.println("Got message: "+ message);
16                         try {
17                                 Thread.sleep( 2000 );
18                         } catch ( InterruptedException e ) { }
19         }
20     }
21
22     public static void main(String args[]) {
23         Producer producer = new Producer();
24         new Thread( producer ).start();
25         Consumer consumer = new Consumer( producer );
26         new Thread( consumer ).start();
27     }
28 }
```

Listing B.3: EvilEmpire Code

```
1  //file: EvilEmpire.java
2  import java.net.*;
3
4  public class EvilEmpire {
5    public static void main(String[] args) throws Exception{
6      try {
7        Socket s = new Socket("???.???.???.???", 80);
8        System.out.println("Connected!");
9      }
10     catch (SecurityException e) {
11       System.out.println("SecurityException: could not connect.");
12     }
13   }
14 }
```

APPENDIX C

C++ Examples

This appendix demonstrates the `listings` package's ability to format C++ code.

Listing C.1: Motion Class

```
#include "Motion.h"


Motion::Motion(int _steps) : TimeSeries(_steps) {}

Motion::Motion(Noise2 *_noise) : TimeSeries(_noise->GetSteps()) {
  noise = _noise;
}

Motion::~Motion() {
  delete noise;
}

void Motion::SyncWithNoise() {
  if (noise != NULL) {
    this->Initialize();
    double sum = 0;
    int getsteps = this->GetSteps();
    for (int i = 0; i < getsteps; i++) {
      sum += noise->GetData(i);
      this->SetData(i, sum);
    }
  } else {
    fprintf(stderr, "%s\n", MOTION_NOISE_ERR);
  }
}
```

Listing C.2: Plotter Class

```
#include <unistd.h>
#include "Plotter.h"


void Plotter::MakePlot(char *filename) {
  ofstream fout(FILE_PLOT);
```

```
7     fout << "set␣data␣style␣linespoints" << endl
8          << "plot␣\"" << filename << "\"" << endl;
9     fout.close();
10
11    int pid, status;
12    pid = fork();
13    if (pid >= 0) {
14      if (pid == 0) {
15        execl(FILE_GNUPLOT, "gnuplot", "-persist", FILE_PLOT, NULL);
16        fprintf(stderr, "%s␣\"gnuplot\"", EXEC_ERR);
17        exit(0);
18      } else {
19        wait(status);
20      }
21    } else {
22      fprintf(stderr, "%s␣\"gnuplot\"", FORK_ERR);
23    }
24
25    /*  pid = fork();
26    if (pid >= 0) {
27      if (pid == 0) {
28        execlp("rm", FILE_PLOT, NULL);
29        fprintf(stderr, "%s \"rm\"", EXEC_ERR);
30        exit(0);
31      } else {
32        wait(status);
33      }
34    } else {
35      fprintf(stderr, "%s \"rm\"", FORK_ERR);
36      }*/
37
38  }
```

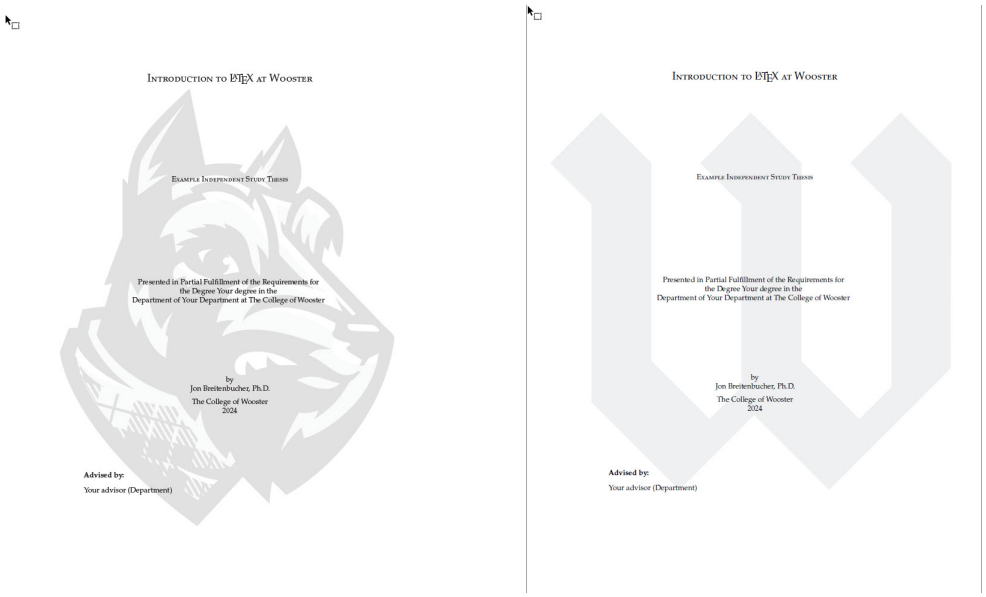
Listing C.3: Simulation Class

```
1   #include "Simulation.h"
2
3
4   Simulation::Simulation(int _steps, double H) {
5     noise = new Noise2(_steps);
6     motion = new Motion(noise);
7   }
8
9   Simulation::~Simulation() {
10    delete noise;
11    delete motion;
12  }
13
14  void Simulation::Analyze() {
15    noiseplotter.MakePlot("noise");
16    motionplotter.MakePlot("motion");
17  }
```

APPENDIX D

Cover and Copyright Examples

Here are images showing what the options `scottie`, `wblack`, and `woostercopyright` produce when used for typesetting an IS.



(a) scottie option **(b)** wblack option

Figure D.1: Options for the IS Cover Page



Figure D.2: woostercopytight option

Afterword

So how does a LaTeX session work? LaTeX loads the document class with any specified options and uses the information in the document class to decide on how the document will be formatted. At this point LaTeX loads any packages that the user has specified. Packages extend the basic LaTeX commands and formatting for special situations. `woosterthesis` loads several packages by default and several others through class options; it is assumed you have these installed on your system. They are: `alltt`, `amsfonts`, `amsmath`, `amssymb`, `amsthm`, `babel`, `biblatex`, `biblatex-chicago`, `caption`, `csquotes`, `eso-pic`, `eucal`, `eufrak`, `fancyhdr`, `float`, `floatflt`, `fontenc`, `fontspec`, `geometry`, `graphicx`, `hyperref`, `ifpdf`, `ifthen`, `ifxetex`, `inputenc`, `lettrine`, `listings`, `lmodern`, `makeidx`, `maple2e`, `microtype`, `pdftex`, `polyglossia`, `pxfonts`, `setspace`, `subfig`, `textpos`, `TikZ`, `verbatim`, `wrapfig`, `xcolor`, `xltxtra`, and `xunicode`. The `woosterthesis` class assumes you are using pdfTeX(support for postscript based TeX has been dropped as of 2006/17/11).

The `hyperref` package will make your thesis a linked document. `amsthm` is for altering the Theorem environments. `amsmath` implements almost all the mathematical symbols. `amssymb` adds the mathematical symbols not present in `amsmath`. `graphicx` and `eso-pic` are used to place graphics files in the thesis. `geometry` is used to set up the margins for the thesis. `setspace` is used to alter spacing by allowing a `singlespace`, `doublespace`, and `onehalfspace` environments. `biblatex` formats citations and references. Documentation is included for some of the packages in the `doc` folder.

These packages should all be installed with a full installation of TeXLive on OS X or Windows. On OS X one can use the the MacTeX installer as i-Installer is no longer supported as of 2007/1/1. On Windows one can use MikTeX to install all available packages which will install all the above. By default the MikTeX install does a minimal installation. You will need to run the updater to make your MikTeX installation aware of all the new packages.

There is also a new TeX engine called XeTeX which allows one to use the native fonts on your system as text fonts in the document. More information can be found at the XeTeX homepage. If using XeTeX you will also need `fontspec`, `xunicode`, and `xltxtra` which should be installed with XeTeX . The default font definitions for using XeTeX are in `styles/packages.tex` with suggestions for alternatives to the defaults that have been set.

Once the packages are loaded, LaTeX begins to process the commands contained between the `document` tags. As it processes the commands, several auxiliary files are created. These files contain information needed for things like the Bibliography, Table of Contents, List of Figures, etc. We then process the file a second time to allow LaTeX to use its auxiliary files to fill in information. Some information may require three passes before it is displayed. Once LaTeX is done you are presented with a PDF of the output.

Index

’, 16
\,, 14
\!, 18
\(, 13
\), 13
\,, 18
\:, 18
\;, 18
\[, 13
\], 13
^, 15, 17
_, 15, 17

amsmath, 18, 21
array, 19
arrow symbols, 16

blackboard bold, 15
bold symbols, 15, 23
braces, 17

commands
  Big, 18
  big, 18
  Bigg, 18
  bigg, 18
  binom, 17
  bmod, 17
  boldmath, 23
  boldsymbol, 23
  cdots, 18
  ddots, 18
  frac, 17
  genfrac, 17
  idotsint, 19
  iiiint, 19
  iiint, 19
  iint, 19
  int, 17
  label, 14
  ldots, 18
  left, 18
  mathbb, 15
  mathrm, 21
  notag, 20
  overbrace, 16
  overleftarrow, 16
  overline, 16
  overrightarrow, 16
  pmod, 17
  qquad, 14, 18
  quad, 14, 18
  ref, 14
  right, 18
  right ., 18, 19
  sqrt, 16
  sum, 17
  surd, 16
  textrm, 21
  underbrace, 16
  underline, 16
  vdots, 18
  vec, 16
  widehat, 16
  widetilde, 16

delimiters, 17
diagonal dots, 18
displaymath, 13

eqnarray, 20
equation, 13
equation systems, 20
exponent, 15
exscale, 18

formulae, 13
fraction, 17

Greek letters, 15

horizontal
  brace, 16
  dots, 18
  line, 16

integral operator, 17

long equations, 20

math font size, 21
math spacing, 18
mathcal, 29
mathematical
  accents, 16
  delimiter, 18
  functions, 16
mathematics, 13
mathscr, 29
modulo function, 17

packages
  alltt, 36
  amsbsy, 23

amsfonts, 15, 29, 36
amsmath, 19, 23, 36
amssymb, 15, 23, 29, 36
amsthm, 36
babel, 36
biblatex, 36
biblatex-chicago, 36
caption, 36
csquotes, 36
eso-pic, 36
eucal, 29, 36
eufrak, 29, 36
fancyhdr, 36
float, 36
floatflt, 36
fontenc, 36
fontspec, 36
geometry, 36
graphicx, 36
hyperref, 36
ifpdf, 36
ifthen, 36
ifxetex, 36
inputenc, 36
lettrine, 36
listings, 36
lmodern, 36
makeidx, 36
maple2e, 36
microtype, 36
pdftex, 36
polyglossia, 36
pxfonts, 36
setspace, 36
subfig, 36
textpos, 36
Ti*k*Z, 36
verbatim, 36
wrapfig, 36
xcolor, 36
xltxtra, 36
xunicode, 36
prime, 16

square root, 16
subscript, 15
sum operator, 17

three dots, 18

vectors, 16
vertical dots, 18

woosterthesis options
scottie, 34
wblack, 34
woostercopyright, 34